



Università degli Studi di Napoli Federico II

Corso di Laurea Triennale in Informatica

Sistema Web per la Gestione e *Storicizzazione* di Referti Clinici

CANDIDATO

Federico Padulano — N86005043

RELATORE

Francesco Isgrò

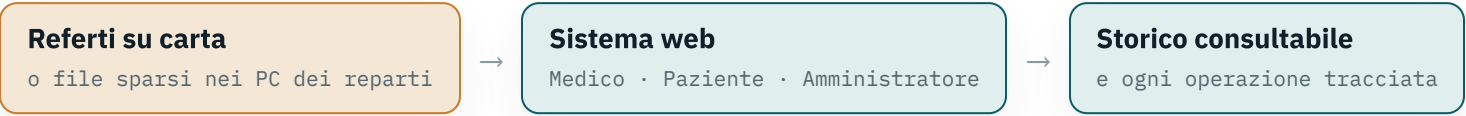
ANNO ACCADEMICO

2025/2026

NOTA PER IL RELATORE

Buongiorno. Presento il progetto sviluppato durante il tirocinio curriculare in Kelyon: un sistema web per la gestione digitale dei referti clinici.

Dalla carta al *digitale*, senza scorciatoie sulla sicurezza



Tirocinio curriculare in Kelyon, con un tutor aziendale: il sistema l'ho sviluppato io, da solo. Software per la sanità digitale dal 2008, certificato ISO 13485.

Vincolo: dati sanitari, quindi obblighi **GDPR** fin dal primo giorno di progettazione.

NOTA PER IL RELATORE

Ho svolto il tirocinio curriculare in Kelyon, affiancato da un tutor aziendale, e il sistema che vi presento l'ho sviluppato io, da solo. Kelyon sviluppa software per la sanità digitale dal 2008, e non parlo di applicazioni gestionali qualsiasi: molti di questi prodotti sono veri e propri Software come Dispositivo Medico, certificati secondo lo standard ISO 13485, che impone processi di sviluppo e controllo qualità più rigidi del normale. In questo contesto la sicurezza non è un dettaglio aggiunto dopo, è un vincolo di progetto fin dall'inizio.

Il problema concreto che mi è stato affidato nasce da una situazione comune negli ospedali: i referti clinici vivono ancora su carta o in file sparsi nei computer dei reparti. Questo significa che un paziente non ha modo di consultare il proprio storico clinico, e non esiste tracciabilità di chi ha caricato cosa e quando — un problema serio non solo per comodità, ma per responsabilità clinica e legale. E trattandosi di dati sanitari, categoria particolare secondo il GDPR, ogni scelta progettuale doveva tenere conto degli obblighi normativi fin dal primo giorno, non aggiungerli come una toppa alla fine.

Tre ruoli, un *flusso* unico

MEDICO

RF3 cerca il paziente per codice fiscale

 →

RF4 carica il referto in PDF

PAZIENTE

RF5/RF7 consulta lo storico, filtrato

RF6 scarica il PDF

RF10 notifica in tempo reale

AMMINISTRATORE

RF2 gestisce gli account medico

RF9 crea gli account paziente

TRASVERSALE · REQUISITI NON FUNZIONALI

RF1/RF8 login sicuro e audit trail

RNF1 sicurezza e privacy

RNF2 integrità dei dati

RNF3 tracciabilità e non ripudio

NOTA PER IL RELATORE

Il sistema ruota attorno a tre ruoli distinti, ognuno con permessi diversi. Il Medico può cercare un paziente inserendo il suo codice fiscale, e una volta trovato, caricare un referto in PDF, associandolo a una categoria e a una data d'esame. Il Paziente ha una propria area personale dove consulta lo storico completo dei referti caricati per lui, lo filtra per categoria o intervallo di date, scarica ogni documento, e riceve una notifica istantanea appena arriva qualcosa di nuovo, se ha la dashboard aperta in quel momento.

L'Amministratore è la figura che governa gli account: crea e disabilita i medici, e soprattutto crea lui stesso gli account dei pazienti, dopo averne verificato di persona l'identità — un paziente non può registrarsi da solo, proprio per garantire che dietro ogni account ci sia una persona verificata. E trasversalmente a tutti i ruoli, ogni accesso richiede un login sicuro, e ogni operazione sensibile viene registrata in un log di audit. Per definire i requisiti non funzionali mi sono appoggiato al modello ISO/IEC 25010 e agli obblighi del GDPR: sicurezza, integrità dei dati e tracciabilità, di cui parlo tra poco più nel dettaglio.

Il rischio nascosto: l'alto *accoppiamento*

70%

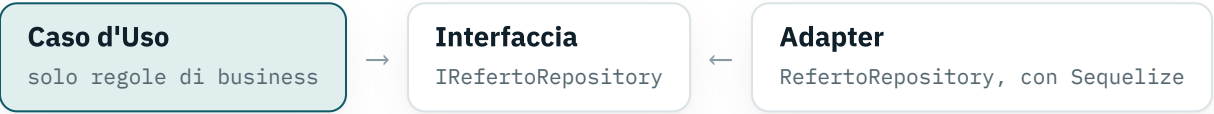
del budget IT delle aziende va oggi al mantenimento dei sistemi esistenti, non a nuove funzionalità

Fonte: Gartner, 2024.

SENZA SEPARAZIONE



CON LA CLEAN ARCHITECTURE



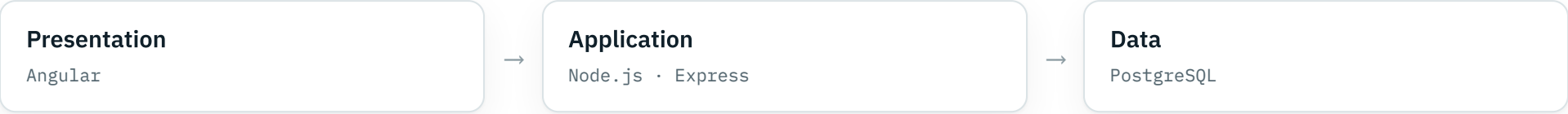
Il 3-Tier decide **dove** stanno i pezzi, non **come** è scritto il codice dentro il server.

NOTA PER IL RELATORE

Un dato che mi ha guidato in questa scelta: secondo Gartner, nel 2024 le aziende spendevano in media il 70% del budget IT solo per mantenere i sistemi esistenti, non per costruirne di nuovi. È esattamente il costo dell'accoppiamento di cui parlo ora. Prima di scrivere codice ho studiato come si organizza un backend. Il 3-Tier da solo è già una buona base per la sicurezza, perché costringe chiunque a passare dal server centrale prima di arrivare ai dati: nessun client parla mai direttamente con il database. Ma non basta a tenere ordinato il codice dentro il server. Il rischio concreto è che, senza una disciplina precisa, si finisca per scrivere nello stesso file, o addirittura nello stesso metodo, sia le regole dell'applicazione — per esempio "un Medico può caricare un referto solo se il paziente esiste" — sia i dettagli tecnici di come si parla con Sequelize per salvarlo sul database.

Quando questi due mondi si mescolano, si crea quello che si chiama alto accoppiamento: il codice diventa aggrovigliato, e cambiare anche un solo dettaglio tecnico, come la libreria di accesso al database o il database stesso, costringerebbe a modificare codice sparso ovunque nel progetto, col rischio di rompere qualcosa che non c'entra nulla con quel cambiamento. Per evitare questo, dentro il server ho applicato la Clean Architecture di Robert Martin, che impone una separazione netta tra le regole di business e i dettagli tecnici.

3-Tier fuori, *Clean Architecture* dentro



DENTRO L'APPLICATION TIER – DAL CENTRO VERSO L'ESTERNO

Frameworks & Drivers
Express · Sequelize · PostgreSQL

Interface Adapters
Controller, Validator – i "traduttori" HTTP ↔ dominio

Use Cases
11 Casi d'Uso – LoginUseCase, CaricaRefertoUseCase...

Entities
Utente, Referto

Regola della Dipendenza: le frecce puntano sempre verso l'interno. Il dominio non sa nulla di Express o Sequelize.

NOTA PER IL RELATORE

La soluzione che ho adottato è la Clean Architecture di Robert Martin: è un modello già esistente, non l'ho inventato io, e il mio lavoro è stato applicarlo a questo progetto. Non sostituisce il 3-tier, ci si inserisce dentro. A livello fisico il sistema è un classico 3-tier: Angular in Presentation, Node ed Express in Application, PostgreSQL in Data. Dentro l'Application Tier, dal centro verso l'esterno: le Entità sono le regole pure del dominio, come Utente e Referto, scritte senza nessuna libreria esterna. I Casi d'Uso sono gli undici moduli che orchestrano le azioni dell'applicazione, uno per ogni operazione — LoginUseCase, CaricaRefertoUseCase, e così via. Gli Adattatori sono i traduttori: i Controller ricevono le richieste HTTP, i Validator ripuliscono i dati in ingresso prima che arrivino ai Casi d'Uso. E all'esterno di tutto, i Dettagli Tecnici: Express, Sequelize, PostgreSQL. La Regola della Dipendenza fa sì che le frecce puntino sempre verso l'interno: il dominio non sa nulla di Express o di Sequelize. Se domani Kelyon cambiasse database, il cuore del sistema non si tocca.

Un referto, dall'HTTP al *database*

●	FRAMEWORKS & DRIVERS
	POST /api/referti
	Express riceve la richiesta multipart; multer tiene il file in memoria come Buffer.
●	INTERFACE ADAPTERS
	RefertiController.upload
	Controlla l'intestazione reale del PDF, cifra il buffer con AES-256-GCM e lo salva su disco — solo ora ottiene percorsoFile.
●	USE CASES
	UploadRefertoUseCase.execute
	Verifica che chi carica sia un Medico, crea l'entità Referto e la salva insieme al log di audit dentro un'unica transazione.
●	ENTITIES
	Referto
	Le regole pure del dominio: richiede sempre un ID, una data e una categoria valide — nessuna libreria esterna.
●	INTERFACE ADAPTERS → FRAMEWORKS & DRIVERS
	RefertoRepository → Sequelize
	Il Caso d'Uso chiede di salvare l'entità attraverso l'interfaccia; solo l'implementazione concreta sa che dietro c'è Sequelize e PostgreSQL.

NOTA PER IL RELATORE

Quello che ho realizzato, in pratica, è questo: per capire davvero come funzionano insieme questi quattro livelli, seguiamo un singolo referto dall'inizio alla fine. La richiesta arriva a Express, che la passa al Controller. Il Controller controlla che il file sia davvero un PDF, lo cifra, lo salva su disco, e solo a quel punto chiama il Caso d'Uso con un input già pulito — solo stringhe, date, ID. Il Caso d'Uso verifica che chi sta caricando sia un Medico, crea l'entità Referto, e la salva insieme al relativo log di audit dentro un'unica transazione. L'entità Referto, nel frattempo, non sa nulla di tutto questo: contiene solo le regole di dominio, come il fatto che una data d'esame non possa essere nel futuro. E quando il Caso d'Uso deve salvare quel Referto, lo fa attraverso l'interfaccia del repository — è solo nell'implementazione concreta, fuori da tutto, che quell'operazione diventa davvero una query Sequelize verso PostgreSQL.

Una cosa importante da chiarire: la Regola della Dipendenza dice che le dipendenze puntano sempre verso l'interno, ma i dati e il controllo della richiesta attraversano i livelli in entrambe le direzioni — entrano dal Controller fino al Caso d'Uso, ed escono di nuovo verso il database attraverso il repository. Quello che non succede mai è che un livello interno importi direttamente il codice di un livello esterno.

Dependency Injection con *tsyringe*

Un Caso d'Uso dichiara di cosa ha bisogno, non sa mai chi gliel'ha fornito.

```
@injectable()
class LoginUseCase {
  constructor(
    @inject('IUtenteRepository') // non Sequelize
    private utenteRepo: IUtenteRepository,
  ) {}
}
```

NOTA PER IL RELATORE

La Regola della Dipendenza dice che un Caso d'Uso non deve mai importare direttamente Sequelize. Ma qualcuno deve pur decidere quale implementazione concreta usare davvero, e passarla quando l'oggetto viene creato — altrimenti la regola resterebbe solo un'intenzione sulla carta. Per questo ho scelto di usare tsyringe, un Dependency Injection Container: una libreria che tiene un registro di quale implementazione concreta corrisponde a quale interfaccia, e costruisce lei stessa ogni oggetto con le sue dipendenze già pronte.

Ogni Caso d'Uso dichiara solo di cosa ha bisogno tramite un decorator @inject, usando il nome dell'interfaccia come token; e in un unico punto del progetto, container.ts, collego ogni interfaccia alla sua implementazione reale — per esempio IUtenteRepository viene collegata alla classe UtenteRepository, che parla davvero con Sequelize. Senza questo meccanismo, ogni Caso d'Uso dovrebbe costruirsi da solo le proprie dipendenze con un new, e proprio lì finirebbe comunque per dipendere da Sequelize: la Regola della Dipendenza si romperebbe nel tentativo stesso di rispettarla.

In pratica, quando un controller chiede al container un'istanza di LoginUseCase, tsyringe legge i suoi @inject, guarda cosa è stato registrato per ciascun token, e costruisce automaticamente ogni dipendenza — comprese le dipendenze delle dipendenze, se ce ne sono — restituendo un LoginUseCase già pronto all'uso. Ed è esattamente questo meccanismo che nei test mi permette di "passare un Fake al posto del vero repository": lo stesso Caso d'Uso, con le stesse due righe di @inject, funziona identico sia con le classi vere sia con le implementazioni finte, perché non sa — e non gli interessa — quale delle due gli è stata data.

Non solo teoria: scelte *verificabili* nel codice

DIPENDERE DA INTERFACCE, NON DA CLASSI CONCRETE



SRP Ogni Caso d'Uso ha un solo motivo per cambiare: LoginUseCase, CambiaPasswordUseCase...

Adapter JwtTokenManager, BcryptPasswordHasher: traducono le librerie esterne nel linguaggio del progetto.

Singleton registerSingleton: una sola istanza di ogni repository per tutta l'app.

NOTA PER IL RELATORE

Queste scelte si appoggiano a principi di progettazione noti in letteratura, quindi non miei: li ho studiati e poi applicati nel codice. Il primo è il Single Responsibility: ogni Caso d'Uso fa una cosa sola, ha un solo motivo per cambiare. LoginUseCase si occupa solo del login, CambiaPasswordUseCase solo del cambio password: se domani cambia una regola sui tentativi di accesso falliti, per esempio, tocco solo il primo file, mai il secondo.

Poi due principi che nascono dalla stessa scelta di fondo, usare interfacce invece di classi concrete: IFileStorage e IPasswordHasher restano ferme, non si toccano mai. Se domani dovessi passare dal disco locale a uno storage su cloud, mi basterebbe scrivere una classe nuova che implementa la stessa interfaccia — il sistema si estende aggiungendo quella classe, senza modificare né l'interfaccia né i Casi d'Uso che la usano oggi. Questo è insieme il Dependency Inversion e l'Open/Closed.

E infine due pattern del catalogo Gang of Four applicati concretamente: il Singleton, nel container che garantisce una sola istanza di ogni repository per tutta l'applicazione; e l'Adapter, in classi come JwtTokenManager o BcryptPasswordHasher, che traducono il vocabolario di una libreria esterna — bcrypt, jsonwebtoken — in quello del mio dominio.

Strumenti scelti per un *team di uno*

BACKEND

 Node.js

 Express

 TypeScript

 Sequelize

 PostgreSQL

tsyringe

 Socket.IO

FRONTEND

 Angular 19

Standalone Components

Signals

RxJS

TESTING

 Jest

Supertest

 Playwright

NOTA PER IL RELATORE

Vi dico cosa ho scelto e perché. Lo stack backend è Node.js con Express in TypeScript: la tipizzazione statica mi ha permesso di individuare a compile-time una classe intera di errori che, su un progetto scritto da solo, altrimenti avrei scoperto solo a runtime. Per la persistenza uso PostgreSQL con Sequelize come ORM, e tsyringe per la dependency injection di cui ho appena parlato. Socket.IO gestisce il canale WebSocket per le notifiche in tempo reale.

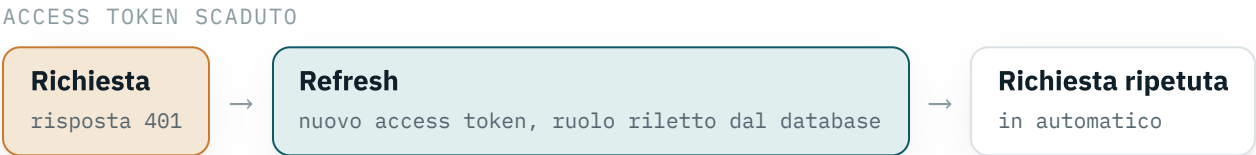
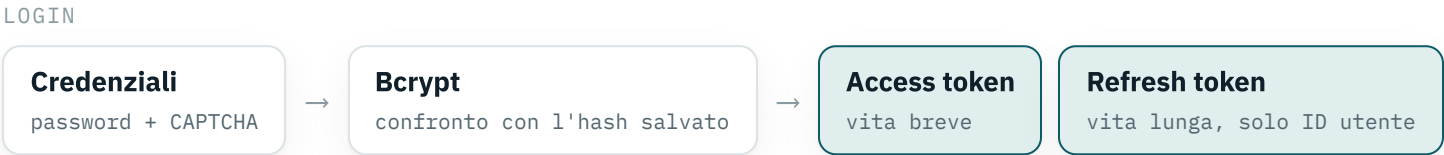
Sul frontend ho scelto Angular 19, più strutturato e opinionato rispetto ad alternative come React: per un'applicazione pensata per crescere e per essere mantenuta anche da altri sviluppatori in futuro, ho preferito una struttura imposta dal framework piuttosto che decisa di volta in volta. Uso i Signal per la reattività, che si integrano meglio con il change detection moderno di Angular rispetto alla sola RxJS. E per la qualità del codice, tre livelli di test — Jest, Supertest, Playwright — di cui parlo tra poco in dettaglio.

Autenticazione: *chi sei* vs cosa puoi fare

\$6,64M

costo medio di una violazione dati nel settore sanitario nel 2026 — il più costoso tra tutti i settori misurati

Fonte: IBM, Cost of a Data Breach Report 2026.



I due token sono firmati con **due segreti distinti**. Al primo accesso il cambio della password provvisoria è obbligatorio.

70%

delle violazioni di dati sanitari indagate dal Garante Privacy nasce dall'invio della documentazione clinica alla persona sbagliata — in metà dei casi, proprio tramite il fascicolo sanitario

Fonte: Garante per la Protezione dei Dati Personali, Relazione Annuale 2019.

Per questo ogni endpoint ricava chi sta chiedendo dal token, mai da un parametro esterno: un Paziente vede solo i referti dove `pazienteId` coincide col proprio profilo, verificato ad ogni richiesta.

NOTA PER IL RELATORE

Prima di entrare nel dettaglio, un numero che inquadra perché ci ho investito tempo: secondo l'IBM Cost of a Data Breach Report 2026, la sanità è il settore con il costo medio più alto per una violazione di dati, 6,64 milioni di dollari, davanti anche ai servizi finanziari. Ho studiato come si gestisce la sicurezza in un'applicazione web e poi l'ho realizzata. Per l'autenticazione ho tenuto separate due cose spesso confuse: chi sei, l'autenticazione, e cosa puoi fare, l'autorizzazione. Le password non sono mai salvate in chiaro: passano da Bcrypt con un salt univoco per ogni utente, che le rende resistenti anche ad attacchi con tabelle precalcolate.

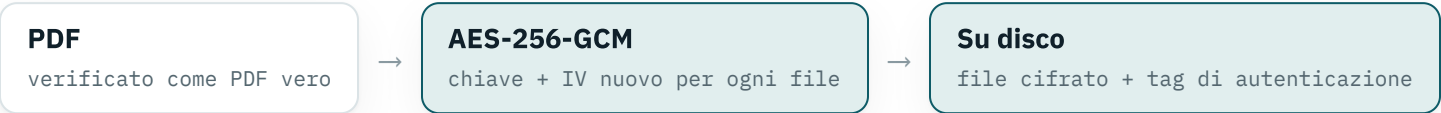
Per la sessione ho usato due token JWT invece di uno solo, e c'è un motivo preciso. Un token che dura poco è più sicuro, perché se venisse rubato la finestra utile a un attaccante sarebbe breve — ma se scadesse troppo spesso, l'utente dovrebbe rifare il login in continuazione. La soluzione è dividere i compiti: l'access token, di breve durata, viaggia con ogni richiesta e autentica le chiamate alle API; il refresh token, più lungo, non si usa mai per accedere alle risorse — serve solo a ottenere un nuovo access token quando quello vecchio scade, senza dover reinserire le credenziali. Sono firmati con due segreti distinti, così anche se uno dei due venisse compromesso, l'altro resta sicuro.

Il refresh token, in particolare, contiene solo l'ID dell'utente, nient'altro: ad ogni rinnovo rileggo l'utente dal database, così un cambio di ruolo o una disabilitazione dell'account si riflettono subito, invece di restare congelati nel vecchio token. E ogni account creato da un Admin deve cambiare la password provvisoria al primo accesso, prima di poter usare qualsiasi altra funzionalità.

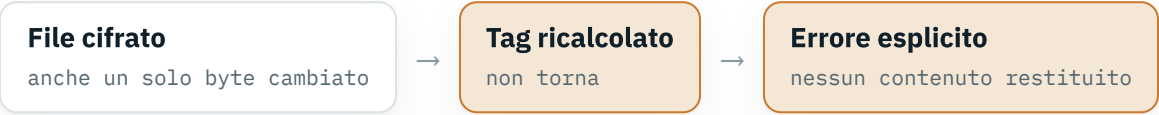
Un dato che mi ha convinto che questo non è un dettaglio: secondo la Relazione Annuale 2019 del Garante Privacy, il 70% delle violazioni di dati sanitari indagate nasce dall'invio della documentazione clinica alla persona sbagliata, e in metà dei casi succede proprio attraverso un fascicolo digitale. Per questo, nel mio sistema, l'identità di chi chiede non arriva mai come parametro dal client: `DownloadRefertoUseCase` confronta l'ID del profilo Paziente ricavato dal token con il `pazienteId` del referto richiesto, e `StoricoRefertiPropriUseCase` ricava il paziente sempre dal token, mai da un ID passato in query. Il rischio che uno veda i referti di un altro non dipende da cosa scrive il client.

I dati a riposo, e la difesa *attiva*

SALVATAGGIO DI UN REFERTO



LETTURA, CON IL FILE MANOMESSO



Login: CAPTCHA e limite di tentativi per indirizzo IP, contro il bruteforce.

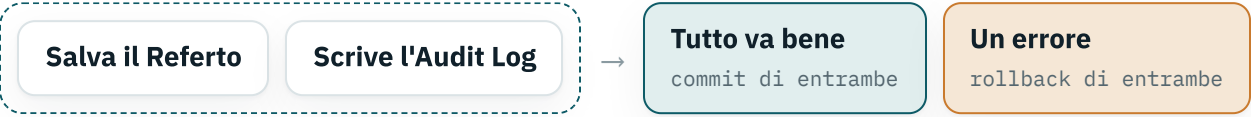
NOTA PER IL RELATORE

Per la cifratura ho scelto uno standard già collaudato, senza inventare niente di mio. I referti sul disco sono cifrati con AES-256-GCM, e vale la pena essere precisi su cosa significa questo nome. AES sta per Advanced Encryption Standard: è l'algoritmo di cifratura simmetrica — la stessa chiave cifra e decifra — diventato lo standard mondiale de facto, scelto dal NIST nel 2001 al termine di una selezione pubblica tra più algoritmi candidati. Il 256 indica la lunghezza della chiave crittografica in bit: più lunga è la chiave, più chiavi possibili esistono, e con 256 bit quel numero è talmente enorme da rendere un attacco a forza bruta — provarle tutte una per una — computazionalmente impraticabile, anche con l'hardware più potente disponibile oggi. GCM, invece, è la modalità con cui l'algoritmo viene usato, ed è cifratura autenticata: oltre a nascondere il contenuto, genera anche un tag di autenticazione che garantisce che il file non sia stato alterato. La modalità più semplice sarebbe stata CBC: cifra e basta, senza accorgersi se qualcuno altera il file dopo — un referto corrotto o manomesso verrebbe decifrato e restituito come se fosse tutto normale, senza nessun errore. Per dei documenti clinici non era un rischio accettabile, ed è questo a giustificare GCM al posto della modalità più semplice. Ogni file riceve inoltre un IV, un vettore di inizializzazione di 12 byte generato in modo casuale ad ogni upload. L'IV non è un segreto, viene salvato in chiaro insieme al file: il suo scopo non è nascondere nulla, ma garantire che cifrare due volte lo stesso identico referto produca due file completamente diversi, così pattern ripetuti nei dati non lasciano tracce riconoscibili nel testo cifrato.

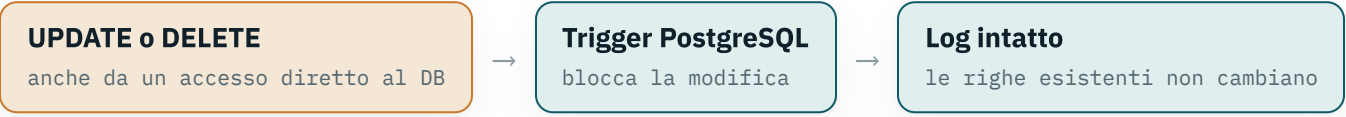
In fase di decifratura, prima di restituire il contenuto in chiaro, il sistema ricalcola il tag di autenticazione e lo confronta con quello salvato: se anche un solo byte del file cifrato è stato alterato dopo il salvataggio, quel confronto fallisce e l'operazione si interrompe con un errore esplicito, invece di restituire silenziosamente un contenuto corrotto o manomesso. La pagina di login, il bersaglio più esposto a tentativi automatizzati, è protetta da un CAPTCHA e da un limite di richieste per indirizzo IP, contro gli attacchi di forza bruta.

Mai dati a metà, mai un log *cancellabile*

RNF2 – TRANSAZIONE ACID



RNF3 – AUDIT LOG IMMUTABILE



NOTA PER IL RELATORE

Qui ho realizzato due requisiti collegati alla fiducia nel sistema. Le operazioni che toccano più tabelle insieme — per esempio salvare un referto e generare il relativo record di log nella stessa operazione — girano dentro transazioni ACID: se una delle scritture fallisce a metà, il database annulla tutto quanto, senza lasciare dati parziali o incoerenti.

Per l'audit log, la parte più delicata non è scriverlo, ma garantirne davvero l'immutabilità. Non basta semplicemente non creare mai una rotta per modificare o cancellare i log: chiunque avesse accesso diretto al database — incluso l'Amministratore, che nel progetto usa le stesse credenziali dell'applicazione — potrebbe comunque alterare o cancellare un log con una semplice query SQL, rendendo inutile tutta la tracciabilità. Per questo ho spostato il vincolo dal livello dell'applicazione al database stesso: un trigger Postgres che blocca qualsiasi tentativo di UPDATE o DELETE sulla tabella dei log, indipendentemente da chi stia eseguendo la query. Ho verificato che funzionasse davvero provando a mandare a mano una query di modifica con le stesse credenziali usate dal server: il database la rifiuta sempre, prima ancora che possa avere effetto.

Il paziente lo sa *subito*, non al prossimo refresh

CON WEBSOCKET



CON IL POLLING, INVECE



Con la connessione aperta non si sprecano richieste, e non c'è ritardo quando arriva qualcosa.

NOTA PER IL RELATORE

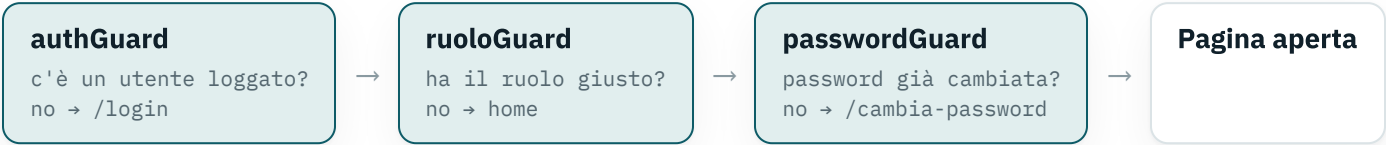
Per le notifiche in tempo reale ho scelto WebSocket con Socket.IO invece del classico polling, e l'ho implementato sia lato server sia lato client. A differenza di una normale richiesta HTTP, che si chiude subito dopo la risposta, una connessione WebSocket resta aperta: è un canale bidirezionale tra client e server, che permette al server di inviare dati al client in qualsiasi momento, senza che il client debba chiedere nulla.

Il paziente, se ha la dashboard aperta, riceve la notifica nell'istante in cui il medico carica un nuovo referto, senza dover ricaricare la pagina. Niente polling: il client non deve interrogare il server a intervalli regolari per sapere se è arrivato qualcosa di nuovo, con tutto lo spreco di richieste inutili e il ritardo che comporterebbe.

L'autenticazione segue lo stesso principio della parte HTTP: alla connessione il client manda il JWT nell'handshake di Socket.IO, e il server lo verifica prima di accettarla — stessa logica dell'autenticazione HTTP, ma sul canale WebSocket. Da lì il server risale dall'utente autenticato al paziente reale, e tiene traccia di quali connessioni attive appartengono a quel paziente: se ha più schede o dispositivi aperti, le riceve tutte. Ed è comunque una consegna best-effort, per onestà: se il paziente non è connesso in quel momento, la notifica va semplicemente persa — non esiste una coda che gliela ripresenti al prossimo accesso, l'unica fonte di verità resta lo storico che ricarica quando riapre la pagina.

Guardie a cascata e *reattività* con i Signal

LE TRE GUARDIE, IN CASCATA, PRIMA DI APRIRE UNA PAGINA PROTETTA



Signal Lo storico dei referti si aggiorna da solo appena arriva un nuovo referto, senza ricaricare la pagina.

NOTA PER IL RELATORE

Sul frontend ho realizzato tre guardie Angular che proteggono le rotte in cascata. Una guardia, in Angular, è una funzione che viene eseguita prima di completare una navigazione: se restituisce falso, la navigazione viene bloccata e l'utente reindirizzato altrove. authGuard verifica che ci sia un token valido, altrimenti reindirizza al login. ruoloGuard controlla che l'utente abbia il ruolo giusto per quella pagina specifica — un Paziente che prova ad aprire l'URL di caricamento referti viene bloccato, anche conoscendo l'indirizzo esatto.

passwordGuard forza il cambio password al primo accesso, leggendo il campo deveCambiarePassword direttamente dal token, prima di lasciar procedere verso qualunque altra pagina protetta. Per la lista dei referti ho sfruttato la stessa reattività in un altro punto: il servizio che ascolta le notifiche via WebSocket tiene un contatore che sale di uno ogni volta che arriva un nuovo referto, e la pagina "Storico Referti" si ricarica da sola non appena quel contatore cambia — non ho dovuto scrivere nessuna logica esplicita per dire "quando arriva una notifica, aggiorna la lista", basta che la pagina osservi quel contatore. Il risultato pratico è che il paziente vede comparire il nuovo referto senza ricaricare la pagina né dover premere nulla.

Le transazioni, senza *violare* la Regola della Dipendenza

Le transazioni sono un concetto di Sequelize. Ma servono dentro i Casi d'Uso, per l'RNf2.

```
// Dentro UploadRefertoUseCase – le due scritture insieme, o
nessuna (RNf2)
await this.gestoreTransazioni.esegui(async (t) => {
  await this.refertoRepo.salva(nuovoReferto, t);
  await this.auditLogRepo.salva(auditLog, t);
});
```

```
// Strato Use Cases – l'astrazione che rende possibile la riga
sopra
interface IGestoreTransazioni {
  esegui<T>(op: (t: unknown) => Promise<T>): Promise<T>;
}
```

```
// Strato Database – l'unica implementazione concreta
class SequelizeGestoreTransazioni implements IGestoreTransazioni {
  async esegui<T>(op: (t: unknown) => Promise<T>): Promise<T> {
    return database.transaction((t) => op(t));
  }
}
```

NOTA PER IL RELATORE

Questa è la scelta più difficile che ho dovuto fare, e vale la pena dire cosa è mio e cosa no: raggruppare più scritture in un'unica transazione è un'idea nota, che assomiglia al pattern Unit of Work, quindi non l'ho inventata io. Mio è il modo in cui l'ho inserita nel sistema: l'interfaccia sta nello strato dei Casi d'Uso e l'implementazione con Sequelize nello strato più esterno, quello dei Frameworks. Un problema concreto che ho dovuto risolvere: le transazioni ACID sono un concetto di Sequelize, ma servono proprio dentro i Casi d'Uso, per garantire l'RNf2. Se importassi Sequelize lì, violerei la Regola della Dipendenza nel punto stesso in cui provo a rispettarla. La soluzione è stata creare un'interfaccia astratta, IGestoreTransazioni, nello strato dei Casi d'Uso: il suo metodo esegui è generico — quella T tra parentesi angolari è un segnaposto, non un tipo vero: TypeScript la sostituisce automaticamente con qualunque tipo restituisca la funzione passata, senza dover scrivere una versione diversa del metodo per ogni caso. Passa poi alla funzione ricevuta in input un gettone che rappresenta la transazione, tipizzato deliberatamente come unknown — una forma senza forma. Il Caso d'Uso sa che quel gettone esiste e lo passa ai repository, ma non sa, e non deve sapere, cosa sia in realtà. Solo nello strato più esterno esiste l'implementazione concreta che la collega a Sequelize.

Concretamente, in UploadRefertoUseCase chiamo gestireTransazioni.esegui passando una funzione che riceve il gettone e lo gira sia al repository del Referto sia a quello dell'Audit Log — così le due scritture condividono la stessa transazione. L'implementazione concreta, SequelizeGestoreTransazioni, si limita a inoltrare quella funzione a database.transaction: da lì in poi è Sequelize a gestire commit e rollback in automatico, se una qualsiasi scrittura al suo interno lancia un errore annulla tutto, altrimenti conferma tutto insieme. Il gettone arriva infine dentro RefertoRepository, ed è solo lì — con un cast esplicito a Transaction — che torna a essere un vero oggetto di Sequelize: è l'unico punto di tutto il progetto dove quel travestimento viene tolto, ed è giusto che sia così, perché è l'unico strato a cui è permesso conoscere Sequelize.

Tre livelli, 76 test automatici

DISTRIBUZIONE DEI 76 TEST, PER LIVELLO



Unit: Fake al posto del DB reale. Integrazione: contro un PostgreSQL reale. End-to-End: browser reale, utente simulato dall'inizio alla fine.

COPERTURA DEL BACKEND



Fake

Implementazione minima ma reale (es. un array in memoria al posto del DB). Nessuna verifica delle interazioni.

```
// non serve un PDF vero: basta l'intestazione "%PDF-" (che il
controller
// verifica) più dei byte riconoscibili da confrontare dopo il round-
trip
const pdfFinto = Buffer.from("%PDF-1.4 contenuto finto per i test");

// ... il medico carica pdfFinto come referto, tramite l'API ...

// Sul disco non deve esserci un PDF in chiaro (l'intestazione "%PDF-"
// non deve comparire): il contenuto è cifrato (AES-256-GCM).
const contenutoSuDisco = fs.readFileSync(percorsoFile);
expect(
  contenutoSuDisco.subarray(0, 5).equals(Buffer.from("%PDF-")),
).toBe(false);

// ... e il download tramite API restituisce il PDF, identico
all'originale ...
const download = await request(app).get(`/api/referti/${id}/download`);
expect((download.body as Buffer).equals(pdfFinto)).toBe(true);
```

NOTA PER IL RELATORE

Ho studiato la piramide dei test e l'ho applicata su tre livelli. Alla base, 34 test unitari, uno per file, uno per ciascuno degli undici Casi d'Uso del sistema, isolati con dei Fake al posto del database vero — implementazioni minime ma reali, come un array in memoria, senza nessuna verifica delle interazioni tra oggetti. Al livello intermedio, 32 test di integrazione che girano contro un PostgreSQL reale, per verificare che i pezzi si parlino davvero e che le query si comportino come previsto.

Il test che mostro qui riguarda la cifratura a riposo (RNF1), ed è un test di integrazione, non unitario: non verifica la funzione di cifratura da sola, verifica l'intera catena. Carica un referto vero tramite l'API, poi legge il file che è stato davvero scritto su disco e controlla che i suoi primi byte non siano l'intestazione "%PDF-", cioè che sul disco non ci sia un PDF leggibile ma solo testo cifrato. Subito dopo scarica lo stesso referto tramite l'API e verifica che i byte tornino identici all'originale, uno per uno. Dimostra le due cose che contano davvero: chi accede al filesystem del server non trova referti in chiaro, ma chi passa dal canale legittimo riottiene il PDF esatto.

L'altro test end-to-end che vale la pena citare pilota due sessioni browser separate contemporaneamente, per verificare che una notifica in tempo reale via WebSocket arrivi davvero da un utente all'altro: cosa che un test con i servizi Angular finti non potrebbe mai dimostrare, perché dietro un mock non c'è nessun WebSocket vero.

La copertura complessiva del backend, misurata sulle istruzioni di codice eseguite dai test, è intorno al 93% (sui rami condizionali scende a circa l'82%), senza aver cercato il 100% apposta: il resto sono soprattutto rami di gestione errori molto specifici, dove il costo di un test dedicato non giustificava il beneficio rispetto a testare il percorso principale.

Tre schermate, *tre requisiti*

medico

Cognome

Neri

esempio.it

esempio.it

Numero matricola

MD-4471

RF2

L'Admin crea un nuovo account Medico, con specializzazione e matricola.

Referti Clinici

Home

Carica referto

Exit

Carica referto

Carica paziente per codice fiscale

CF17861975312330

La ricerca parte da sola appena il codice fiscale è completo.

Paziente trovato

NOME

Mario Rossi

COGNOME E FISCALE

CF178619753123308

DATA DI NASCITA

01/08/1985

Carica un nuovo referto

Categoria

Infettivologia

02/2024

Data esame

erti

Dal

dd/mm/yyyy

Al

dd/mm/yyyy

Data esame

Data caricamento

15/07/2026

08/08/2026 16:07

15/07/2026

08/08/2026 16:06

Pagina 1 di 1

NOTA PER IL RELATORE

Ecco il risultato di quello che ho realizzato, con qualche schermata: la creazione di un account medico da parte dell'Amministratore, il caricamento di un referto da parte del Medico dopo la ricerca per codice fiscale, e lo storico del Paziente con i filtri per categoria e intervallo di date.

Cosa è stato *raggiunto*

Le dieci funzionalità richieste, sui tre ruoli previsti. Problema per problema, cosa cambia davvero:

PRIMA	CON QUESTO SISTEMA
Referti su carta o file sparsi: nessuno storico consultabile dal paziente	Storico digitale, filtrabile per categoria e data, aggiornato in tempo reale via WebSocket
70% delle violazioni dati sanitari (Garante, 2019): referto arrivato alla persona sbagliata	Accesso per ruolo verificato ad ogni richiesta: un Paziente vede solo i propri referti, mai quelli di un altro
Nessuna tracciabilità di chi ha caricato o modificato un referto	Audit log immutabile: un trigger PostgreSQL blocca UPDATE e DELETE, non solo l'applicazione
Un file su disco, se raggiunto, è leggibile da chiunque vi acceda	Referti cifrati con AES-256-GCM prima del salvataggio: senza la chiave, il contenuto è inutilizzabile

Verificato da 76 test automatici su tre livelli, ≈93% delle istruzioni del backend eseguite dai test. Database, file system e transazioni restano sostituibili senza toccare i casi d'uso, che dipendono solo da interfacce.

NOTA PER IL RELATORE

Il sistema copre tutti i requisiti funzionali assegnati, sui tre ruoli previsti, con un'attenzione particolare alla protezione dei dati: i documenti sono cifrati prima di essere salvati su disco, l'autenticazione usa un access token e un refresh token, e il log di audit è protetto da un trigger a livello di database, non solo di applicazione. Non ho invece configurato HTTPS, che in produzione andrebbe messo davanti al server. La qualità del codice è verificata con tre livelli di test automatici indipendenti, per un totale di 76 test e una copertura del backend intorno al 93% delle istruzioni. Sul lato architettura, i casi d'uso conoscono solo interfacce come IFileStorage, i repository e IGestoreTransazioni: cambiare il database o dove vengono salvati i file vuol dire riscrivere un adapter, non la logica di business. I test unitari lo mostrano, perché eseguono i casi d'uso con repository finti al posto di quelli reali.

Cosa si può ancora *aggiungere*

- Storage su cloud (S3/MinIO) — già isolato dietro un'interfaccia, basta una classe nuova
- Paginazione lato server per lo storico, quando i volumi crescono
- Notifiche anche via email, per quando il paziente non ha la dashboard aperta

NOTA PER IL RELATORE

Alcuni sviluppi futuri naturali: lo storage dei file è già isolato dietro un'interfaccia, quindi passare da disco locale a uno storage cloud come S3 richiederebbe di scrivere una sola nuova classe, senza toccare il resto del sistema. Poi la paginazione lato server per lo storico, quando i volumi crescono molto. E le notifiche via email, per quando il paziente non ha la dashboard aperta nel browser.

Grazie per *l'attenzione*

Sono a disposizione per le domande.

NOTA PER IL RELATORE

Grazie per l'attenzione, sono a disposizione per le domande.